Hands-on Quantum Magnetometry Using Helmholtz Coils and Arduino-Controlled Currents

Abstract

This project investigates the practical aspects of magnetic field generation and quantum magnetometry using a three-axis Helmholtz coil system driven by an Arduino microcontroller. The setup is used to simulate both static and time-varying magnetic fields, which are measured using a diamond nitrogen-vacancy (NV) center-based magnetometer. Through this implementation, we demonstrate the interplay between classical electromagnetic theory and quantum-level sensing, offering insights into sensor calibration, magnetic field uniformity, and programmable control of current flow.

Introduction

Magnetic fields are ubiquitous, playing crucial roles in natural systems and engineered devices. Accurately measuring these fields opens doors to a range of high-impact applications including GPS-independent navigation, geophysical exploration, and quantum computing. One of the most promising approaches to sensitive field detection is quantum magnetometry, which leverages quantum mechanical phenomena such as spin-dependent fluorescence in atomic-scale systems.

Among the most prominent quantum magnetometers is the nitrogen-vacancy (NV) center in diamond, a lattice defect that serves as a stable quantum sensor. Under the influence of a magnetic field, the NV center exhibits shifts in its energy levels, detectable via laser-induced fluorescence. While this detection requires sophisticated optics and photodetectors, the prerequisite to any quantum magnetic measurement is a controlled, uniform, and tunable magnetic field environment.

To establish such an environment, we developed a system comprising a programmable Arduino, motor drivers, and a 3-axis Helmholtz coil, allowing for spatial and temporal control over magnetic field generation.

Helmholtz Coils and Magnetic Field Theory

A Helmholtz coil is a configuration of two identical circular coils, placed symmetrically along a common axis and separated by a distance equal to their radius. When current flows through both coils in the same direction, the superimposed magnetic fields combine to form a highly uniform field at the midpoint.

The magnetic field B at the center of a Helmholtz coil pair is given by the Biot–Savart Law:

$$B = \frac{8\mu_0 N I}{\sqrt{125}\, R}$$

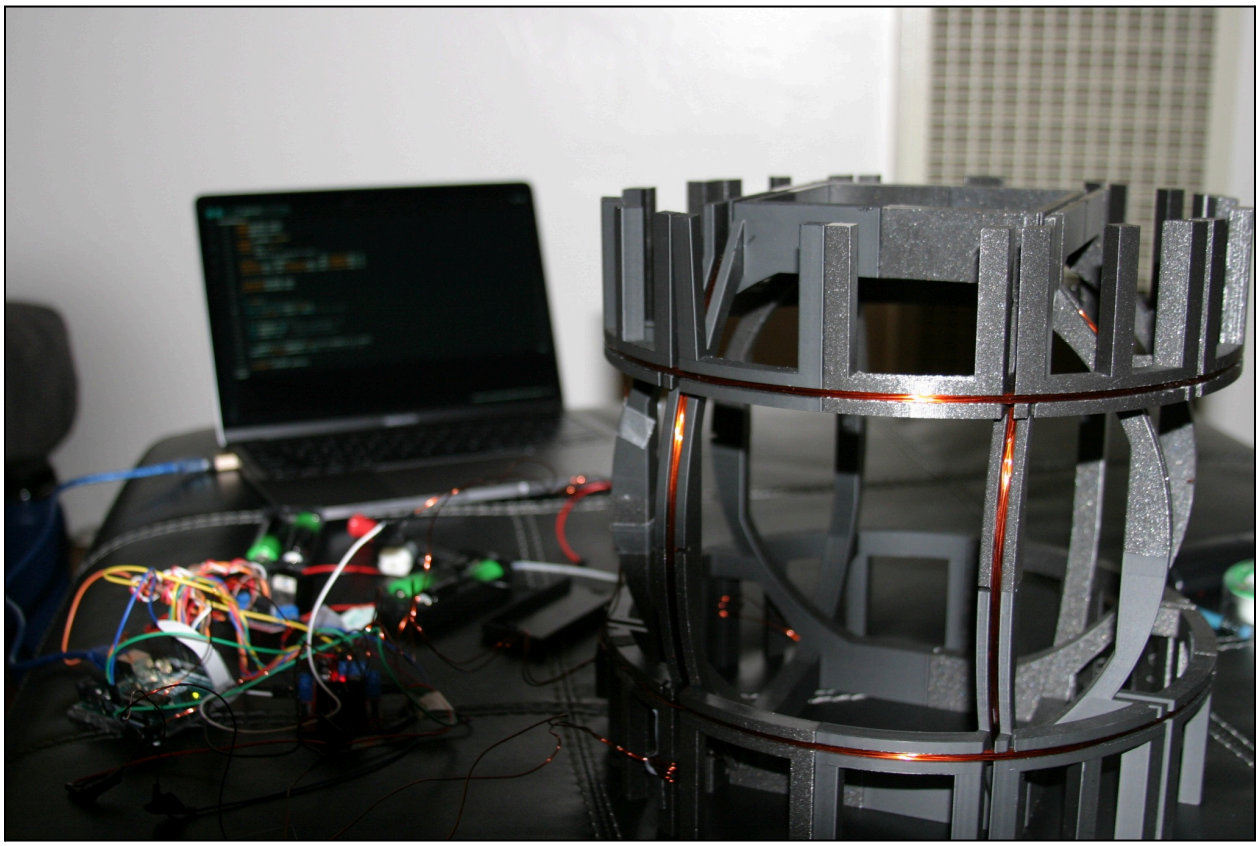
Where:

- μ_0: permeability of free space,
- N: number of turns in each coil,
- I: current through the coil,
- R: radius of the coil.

In our design, each coil pair consists of approximately 10 turns and a radius of 0.178 meters (14 inches in diameter). These parameters allow for adjustable field strengths by simply modifying the current using PWM control from the Arduino.

Our project includes a three-axis Helmholtz coil, enabling independent generation of magnetic fields along the X, Y, and Z axes. This capability is essential for full vector field control in experiments such as magnetometer calibration.

System Architecture and Circuit Design



The electrical control system is centered around an Arduino Uno, which acts as the programmable controller. The Arduino interfaces with two L298N dual H-bridge motor drivers, which in turn supply current to the Helmholtz coils and a DC motor used for validation.

Each driver module handles two channels:

- Driver 1: Z coil and motor
- Driver 2: X and Y coils

To regulate the magnetic field strength, PWM (Pulse Width Modulation) signals are generated by the Arduino. These signals vary the average current delivered to each coil, thereby adjusting the resulting magnetic field in a controlled manner.

The direction of the current is determined by setting digital HIGH and LOW values on the respective input pins. An external DC power source is used to supply sufficient current beyond what the Arduino alone can provide.

The circuit was verified in stages, beginning with standalone motor testing to confirm proper PWM operation. Once successful, the circuit was extended to all three coils, and final integration with the NV magnetometer was completed.

Experimental Modes and Methodology

To investigate the behavior of static and time-varying magnetic fields, we implemented two primary operating modes using the three-axis Helmholtz coil system. In addition, preliminary tests involving a DC motor were conducted to validate circuit functionality before magnetometer integration.

4.1 Preliminary Mode: Motor-Based Validation(Non-Experimental)

Before connecting the magnetometer and coils, we used a DC motor as a diagnostic load to verify that our PWM signals and H-bridge motor drivers were functioning correctly. This testing ensured that:

- PWM signals from the Arduino modulated current as expected,
- Current direction was properly controlled via digital logic,
- The motor drivers could reliably deliver variable current from an external power supply.

These tests were not part of the core magnetic field experiments, but were crucial to establish hardware readiness.

4.2 Core Experimental Modes

After validating the circuit, we conducted the actual magnetic field experiments using the Helmholtz coil system. The following modes were implemented:

4.2.1 Constant Current to All Coils

To establish controlled, uniform magnetic fields, we applied fixed PWM values to all three axes. The values tested were 85, 170, and 255 (on a scale of 0–255), corresponding to increasing current magnitudes. As expected:

- Lower PWM (85) resulted in weaker and less stable field measurements,
- Higher PWM (170–255) produced stronger and more stable fields with reduced noise.

This mode enabled calibration and comparative analysis across different field strengths.

4.2.2 Sinusoidal Current to All Coils

A low-frequency (0.2 Hz) sinusoidal PWM waveform was applied to all three coil axes via their respective EN pins. This generated a time-varying magnetic field with a smooth, periodic profile, effectively simulating a low-frequency alternating magnetic field. This mode is particularly useful for testing how quantum sensors like NV centers respond to changing magnetic environments over time.

Arduino Code Implementation

The system was controlled using a set of Arduino programs tailored for each test mode. These programs defined the direction and strength of current through each coil and the motor via digital pin controls and analogWrite() for PWM.

The core structure includes:

- Pin setup for EN (PWM), IN1, IN2 for all outputs
- Forward direction set via HIGH/LOW pairs
- PWM modulation implemented in the loop() function (either constant or sinusoidal)
- The complete code is provided in the appendix of the full report.

Results and Interpretation

The magnetometer data collected under varying current conditions confirmed the theoretical expectations:

- Field Strength vs. PWM: Higher PWM values led to proportionally stronger magnetic fields.
- Stability: At lower PWM levels (e.g., 85), magnetic field readings were more variable. At higher PWM (170–255), the fields were stronger and more uniform, as predicted by Biot–Savart.
- Sinusoidal Field Behavior: When a time-varying current was applied, the magnetometer readings showed periodic fluctuations consistent with the applied waveform.

This verified that our circuit and code architecture provided effective and predictable control over magnetic field strength and behavior.

NOTE: Please view the accompanying video for this project to see the actual data, as it was only recorded in video format.

Conclusion

This project successfully demonstrated the design and execution of a three-axis magnetic field generator using a Helmholtz coil system controlled via Arduino. Through both constant and sinusoidal current profiles, we were able to validate theoretical predictions using a diamond-based quantum magnetometer. The experiment bridges classical electromagnetic design with modern quantum sensing.

References

1. *How to control a DC motor with an Arduino - projects*. All About Circuits. (n.d.). https://www.allaboutcircuits.com/projects/control-a-motor-with-an-arduino/

2. *L298N motor driver Arduino: Motors: Motor Driver: L298n*. Arduino Project Hub. (n.d.). https://projecthub.arduino.cc/lakshyajhalani56/l298n-motor-driver-arduino-motors-motor-driver-l298n-7e1b3b

3. Wikimedia Foundation. (2025a, February 11). *Helmholtz coil*. Wikipedia. https://en.wikipedia.org/wiki/Helmholtz_coil

4. Wikimedia Foundation. (2025, March 28). *Biot–Savart law*. Wikipedia. https://en.wikipedia.org/wiki/Biot%E2%80%93Savart_law

Appendix

a. Constant Current Code

```
// This code should run a constant current through every driver output including the motor and the coil.

// === Pin Definitions ===

// Driver 1 (Motor and Z coil)
const int IN1_MOTOR = 7;
const int IN2_MOTOR = 4;
const int EN_MOTOR  = 6;

const int IN1_Z = 3;
const int IN2_Z = 2;
const int EN_Z  = 5;

// Driver 2 (X and Y coils)
const int IN1_X = 12;
const int IN2_X = 13;
const int EN_X  = 11;

const int IN1_Y = 9;
const int IN2_Y = 8;
const int EN_Y  = 10;

void setup() {
 // Setup all pins as outputs
 pinMode(IN1_X, OUTPUT); pinMode(IN2_X, OUTPUT); pinMode(EN_X, OUTPUT);
 pinMode(IN1_Y, OUTPUT); pinMode(IN2_Y, OUTPUT); pinMode(EN_Y, OUTPUT);
 pinMode(IN1_MOTOR, OUTPUT); pinMode(IN2_MOTOR, OUTPUT); pinMode(EN_MOTOR, OUTPUT);
 pinMode(IN1_Z, OUTPUT); pinMode(IN2_Z, OUTPUT); pinMode(EN_Z, OUTPUT);

 // Set direction for all outputs (forward)
 digitalWrite(IN1_X, HIGH); digitalWrite(IN2_X, LOW);
 digitalWrite(IN1_Y, HIGH); digitalWrite(IN2_Y, LOW);
 digitalWrite(IN1_Z, HIGH); digitalWrite(IN2_Z, LOW);
 digitalWrite(IN1_MOTOR, HIGH); digitalWrite(IN2_MOTOR, LOW);

 // Constant PWM value for all outputs
 int constantPWM = 85;  // Adjust this value (0-255) as desired we used 85 170 and 255
```

```
 analogWrite(EN_X, constantPWM);
 analogWrite(EN_Y, constantPWM);
 analogWrite(EN_Z, constantPWM);
 analogWrite(EN_MOTOR, constantPWM);
}

void loop() {
 // Nothing needed - outputs are constant
}
```

b. Sinusoidal Current Code

```
// This code should run a sinusoidal current through every driver output including the motor and the coil.


// === Pin Definitions ===

// Driver 1 (Motor and Z coil)
const int IN1_MOTOR = 7;
const int IN2_MOTOR = 4;
const int EN_MOTOR  = 6;

const int IN1_Z = 3;   // Z coil IN3 on driver (confirmed)
const int IN2_Z = 2;   // Z coil IN4 on driver (confirmed)
const int EN_Z  = 5;   // ENB for Z coil

// Driver 2 (X and Y coils)
const int IN1_X = 12;  // X coil IN1
const int IN2_X = 13;  // X coil IN2
const int EN_X  = 11;  // ENA for X coil

const int IN1_Y = 9;   // Y coil IN3
const int IN2_Y = 8;   // Y coil IN4
const int EN_Y  = 10;  // ENB for Y coil

void setup() {
 // Setup all pins as outputs
 pinMode(IN1_X, OUTPUT); pinMode(IN2_X, OUTPUT); pinMode(EN_X, OUTPUT);
 pinMode(IN1_Y, OUTPUT); pinMode(IN2_Y, OUTPUT); pinMode(EN_Y, OUTPUT);
 pinMode(IN1_MOTOR, OUTPUT); pinMode(IN2_MOTOR, OUTPUT); pinMode(EN_MOTOR, OUTPUT);
 pinMode(IN1_Z, OUTPUT); pinMode(IN2_Z, OUTPUT); pinMode(EN_Z, OUTPUT);

 // Set direction for all outputs (forward)
 digitalWrite(IN1_X, HIGH); digitalWrite(IN2_X, LOW);
 digitalWrite(IN1_Y, HIGH); digitalWrite(IN2_Y, LOW);
 digitalWrite(IN1_Z, HIGH); digitalWrite(IN2_Z, LOW);
 digitalWrite(IN1_MOTOR, HIGH); digitalWrite(IN2_MOTOR, LOW);
}

void loop() {
 // Sinusoidal modulation for all outputs
```

```
 static unsigned long tStart = millis();
 float t = (millis() - tStart) / 1000.0; // Time in seconds

 // Parameters for sine wave
 float frequency = 0.2;  // 0.2 Hz = 1 cycle every 5 sec
 float amplitude = 127;  // Half of 255 (max PWM)
 float offset = 127;     // Center point of sine wave

 // Calculate sinusoidal PWM value
 int pwmValue = (int)(amplitude * sin(2 * PI * frequency * t) + offset);
 pwmValue = constrain(pwmValue, 0, 85); // Ensure within PWM range 85, 170, 255

 // Apply PWM to all outputs
 analogWrite(EN_X, pwmValue);
 analogWrite(EN_Y, pwmValue);
 analogWrite(EN_Z, pwmValue);
 analogWrite(EN_MOTOR, pwmValue);
}
```

c. Constant Current Code for Motor Only

```
// This code should run a constant current only through the motor.

// === Pin Definitions ===

// Driver 1 (X coil)
const int IN1_X = 12;
const int IN2_X = 13;
const int EN_X  = 11;

// Driver 1 (Y coil)
const int IN1_Y = 7;
const int IN2_Y = 5;
const int EN_Y  = 10;

// Driver 2 (Motor)
const int IN1_MOTOR = 9;
const int IN2_MOTOR = 8;
const int EN_MOTOR  = 6;

// Driver 2 (Z coil)
const int IN1_Z = 4;
const int IN2_Z = 3;
const int EN_Z  = 2;

void setup() {
 pinMode(IN1_X, OUTPUT);
 pinMode(IN2_X, OUTPUT);
 pinMode(EN_X, OUTPUT);

 pinMode(IN1_Y, OUTPUT);
 pinMode(IN2_Y, OUTPUT);
 pinMode(EN_Y, OUTPUT);

 pinMode(IN1_MOTOR, OUTPUT);
 pinMode(IN2_MOTOR, OUTPUT);
 pinMode(EN_MOTOR, OUTPUT);

 pinMode(IN1_Z, OUTPUT);
 pinMode(IN2_Z, OUTPUT);
 pinMode(EN_Z, OUTPUT);
```

```
 digitalWrite(IN1_X, LOW);
 digitalWrite(IN2_X, LOW);
 digitalWrite(IN1_Y, LOW);
 digitalWrite(IN2_Y, LOW);
 digitalWrite(IN1_Z, LOW);
 digitalWrite(IN2_Z, LOW);

 analogWrite(EN_X, 0);
 analogWrite(EN_Y, 0);
 analogWrite(EN_Z, 0);

 digitalWrite(IN1_MOTOR, HIGH);
 digitalWrite(IN2_MOTOR, LOW);
 analogWrite(EN_MOTOR, 180);
}

void loop() {
 // Nothing here - motor runs continuously, coils are held off.
}
```

d. Sinusoidal Current Code for Motor Only

```
// This code should run a sinusoidal current only through the motor.

// === Pin Definitions ===

// Driver 1 (X coil)
const int IN1_X = 12;
const int IN2_X = 13;
const int EN_X  = 11;

// Driver 1 (Y coil)
const int IN1_Y = 9;
const int IN2_Y = 8;
const int EN_Y  = 10;

// Driver 2 (Motor)
const int IN1_MOTOR = 7;
const int IN2_MOTOR = 4;
const int EN_MOTOR  = 6;

// Driver 2 (Z coil)
const int IN1_Z = 3;
const int IN2_Z = 2;
const int EN_Z  = 5;

void setup() {
 // Setup all pins
 pinMode(IN1_X, OUTPUT);
 pinMode(IN2_X, OUTPUT);
 pinMode(EN_X, OUTPUT);

 pinMode(IN1_Y, OUTPUT);
 pinMode(IN2_Y, OUTPUT);
 pinMode(EN_Y, OUTPUT);

 pinMode(IN1_MOTOR, OUTPUT);
 pinMode(IN2_MOTOR, OUTPUT);
 pinMode(EN_MOTOR, OUTPUT);

 pinMode(IN1_Z, OUTPUT);
 pinMode(IN2_Z, OUTPUT);
```

```
 pinMode(EN_Z, OUTPUT);

 // Set coil directions and disable them (no current)
 digitalWrite(IN1_X, LOW); digitalWrite(IN2_X, LOW); analogWrite(EN_X, 0);
 digitalWrite(IN1_Y, LOW); digitalWrite(IN2_Y, LOW); analogWrite(EN_Y, 0);
 digitalWrite(IN1_Z, LOW); digitalWrite(IN2_Z, LOW); analogWrite(EN_Z, 0);

 // Set motor direction
 digitalWrite(IN1_MOTOR, HIGH);
 digitalWrite(IN2_MOTOR, LOW);
}

void loop() {
 // Sinusoidal modulation for motor
 static unsigned long tStart = millis();
 float t = (millis() - tStart) / 1000.0; // Time in seconds

 // Parameters for sine wave
 float frequency = 0.2;  // 0.2 Hz = 1 cycle every 5 sec
 float amplitude = 127;  // Half of 255 (max PWM)
 float offset = 127;     // Center point of sine wave

 // Calculate sinusoidal PWM value
 int pwmValue = (int)(amplitude * sin(2 * PI * frequency * t) + offset);
 pwmValue = constrain(pwmValue, 0, 180); // Ensure within PWM range

 // Apply PWM to motor
 analogWrite(EN_MOTOR, pwmValue);
}
```